

# Urban Mobility Optimization System (UMOS): Robust System Design and Project Management Plan

Natalie Mariño Figueroa – 20232020143, nmarinof@udistrital.edu.co<sup>1</sup>, Edison Santiago Sepúlveda Cortés – 20231020237, essepulvedac@udistrital.edu.co<sup>1</sup>, Marlon Yecid Riveros Guio – 20231020208, myriverosg@udistrital.edu.co<sup>1</sup>, and Juan Esteban Cañon Solorza – 20232020078, jecanons@udistrital.edu.co<sup>1</sup>

<sup>1</sup>Dept. of Computer Engineering, Universidad Distrital Francisco José de Caldas

## ABSTRACT

This document presents the third and final workshop deliverable for the Urban Mobility Optimization System (UMOS) project. It builds on the systems analysis from Workshop #1 and the architectural design from Workshop #2 by adding two layers that were missing: robust engineering practices grounded in quality standards (ISO 9001, CMMI, IEEE 730), and a complete project management plan following PMBOK and ISO 31000 frameworks. The document traces how the design evolved across three workshops, what quality mechanisms we added and why, how we structured the risk management around the sensitivity findings from our data collection, and how the 20-week implementation is organized in terms of team roles, milestones, budget, and stakeholder communication. The goal is to turn what was an analytical exercise and a paper design into something that could actually be built and deployed.

**Keywords:** Robust design, quality assurance, risk management, project management, ISO 9001, PMBOK, urban mobility, UMOs

## I. EXECUTIVE SUMMARY

This is the third document in a series of three. The first workshop produced a systems analysis of the transport network serving the Universidad Distrital in Bogotá: 24 elements cataloged, 312 surveys collected, 20 hours of observation data, 4 feedback loops identified, 6 bottlenecks mapped, and an overall performance score of 12/25 that confirmed what students already knew from experience. The second workshop translated those findings into a software design: a microservices architecture with seven modules, five UML diagrams, a technology stack justified against the data, an Ant Colony Optimization algorithm for routing, and a 20-week implementation roadmap.

This third workshop asks a different question. Not “what should we build?” or “how should we build it?” but “how do we make sure it actually works, and how do we manage the process of building it?” That meant adding two things the previous workshops had not addressed in depth: a quality assurance framework tied to recognized standards, and a project management plan with real roles, real timelines, and real budgets.

The core additions are: (1) architectural refinements guided by ISO 9001 quality principles and IEEE 730 software quality assurance, (2) a risk management plan following ISO 31000 and PMBOK that maps directly onto the sensitivity parameters we measured in Workshop #1, (3) quality metrics with acceptance criteria for every major module, (4) a project charter defining scope, stakeholders, and success criteria, (5) a team structure with accountability assignments, (6) a Gantt-based timeline with critical path analysis, and (7) a resource and communication plan.

## II. DESIGN EVOLUTION: FROM ANALYSIS TO ROBUST DESIGN

### A). Workshop #1 to #2: From Problem to Architecture

The first workshop told us that the transport system is complex in the technical sense: feedback loops amplify small disturbances (bus bunching, rain cascades), emergent behaviors escape central control (WhatsApp coordination, spontaneous queuing), and the most sensitive parameter is not traffic volume but rainfall intensity (+26.4% wait time per 10% change). That understanding pushed us away from a static routing tool and toward an adaptive system with weather as a first-class input and ACO as the routing algorithm.

Workshop #2 translated those insights into architecture. Seven NestJS microservices, each mapping to a subsystem identified in the analysis. Five UML diagrams formalizing the design. A deployment model on GKE with auto-scaling tied to the demand patterns we measured. Requirements split into functional (8), non-functional (6), and domain-specific (2, covering equity and scalability).

### B). Workshop #2 to #3: From Architecture to Robustness

The jump from Workshop #2 to #3 was less dramatic but arguably more important for actually shipping something. Three gaps became apparent when we reviewed the W2 design:

First, we had described *what* the system should do under normal conditions but not how it should *fail*. The risk management in W2 was limited to five bullet points about API failures. This workshop expands that into a full ISO 31000-aligned frame-

work with risk registers, probability-impact matrices, and contingency protocols for each failure scenario.

Second, we had set performance targets (99.5% uptime, 2-second route calculation) but had not defined how we would *verify* them. This workshop adds a quality assurance framework with acceptance criteria, testing strategies, and monitoring mechanisms for each module.

Third, we had a 20-week roadmap but no project management structure. No defined roles. No accountability matrix. No budget breakdown. No communication plan. This workshop fills those gaps.

### C). *Lessons Learned Across Three Workshops*

A few things changed in how we think about this project:

We initially underestimated the importance of weather data. The sensitivity analysis in W1 forced us to make it the top sprint priority in W2, and in W3 we added a dedicated quality metric for weather data freshness (target: 95% of readings under 5 minutes old).

We initially assumed the routing algorithm was the hard part. It is not. The hard part is data integration: getting five external APIs to feed reliable, timely data into the system when none of them offer uptime guarantees. The W3 risk analysis reflects this by giving API dependency the highest combined risk score.

We initially treated the equity requirement as a nice-to-have. The W1 data showing a  $3.4\times$  cost-per-km disparity made it non-negotiable. In W3, we added equity-related quality metrics and acceptance criteria that treat it as a first-class system objective.

## III. ROBUST ARCHITECTURE DESIGN

### A). *Quality Standards Applied*

We applied three quality frameworks, each for a different reason:

**ISO 9001:2015** for the overall quality management system. Specifically, we adopted its Plan-Do-Check-Act (PDCA) cycle for the development process, its emphasis on customer focus (mapped to our stakeholder analysis from W1), and its requirement for documented evidence of quality objectives. We did not pursue formal certification (impractical for a student project), but we structured our processes to align with ISO 9001 clauses 4 through 10 [7].

**IEEE 730 (Software Quality Assurance Plans)** for the software-specific quality framework. This standard defines what a quality assurance plan should contain: purpose, scope, referenced documents, management responsibilities, documentation requirements, standards and practices, reviews, testing, and problem resolution. Our QA framework in Section IV follows this structure [8].

**CMMI Level 2 (Managed)** as a process maturity target. At Level 2, the key process areas are requirements management, project planning, project monitoring, configuration management, and quality assurance. All five are addressed in this document. We are not claiming CMMI appraisal, but we used the framework to check that we were not missing obvious pro-

cess gaps [9].

### B). *Architectural Refinements*

Three changes to the W2 architecture resulted from applying these standards:

**Fault isolation boundaries.** In W2, we described microservices but did not define explicit failure domains. Now each module has a defined blast radius: if the Notification Service crashes, it affects only push alerts, not route calculation. If the Weather Data Adapter returns stale data, the system flags it to the user (data age display) rather than silently serving outdated recommendations. This aligns with ISO 9001's requirement for risk-based thinking in process design.

**Configuration management.** W2 specified GitHub Actions + ArgoCD for CI/CD but did not address configuration drift, secrets management, or environment parity. We added Terraform for infrastructure-as-code (preventing drift between staging and production), Vault for secrets rotation, and a promotion pipeline that requires passing integration tests in staging before production deployment.

**Audit trails and traceability.** ISO 9001 clause 7.5 requires documented information for evidence of conformity. We added structured logging (via Loki) with correlation IDs across microservices, and a decision log that records every architectural change with its rationale and the W1/W2 finding that motivated it.

### C). *Modularity, Scalability, and Maintainability*

The seven-module architecture from W2 remains unchanged in its logical structure. What changed is how we documented the quality attributes of each module:

**Table 1:** Module Quality Attributes

| Module           | Availability | Latency | Failure Mode             |
|------------------|--------------|---------|--------------------------|
| Route Engine     | 99.5%        | <2s     | Degrade to cached routes |
| Data Integration | 99.0%        | <1s     | Serve stale data + flag  |
| User Management  | 99.9%        | <200ms  | Offline auth cache       |
| Notifications    | 98.0%        | <5s     | Queue and retry          |
| Traffic Monitor  | 99.0%        | <30s    | Static schedule fallback |
| Persistence      | 99.9%        | <100ms  | Read replica failover    |
| Policy Engine    | 99.9%        | <50ms   | Default policy set       |

## IV. QUALITY ASSURANCE FRAMEWORK

### A). *Quality Objectives*

Following IEEE 730, we define measurable quality objectives for the UMOS:

**Table 2:** Quality Objectives and Acceptance Criteria

| Objective         | Metric                   | Target                       | Measurement          |
|-------------------|--------------------------|------------------------------|----------------------|
| Route accuracy    | ETA vs actual time       | $\pm 3$ min for 80% of trips | Post-trip comparison |
| Data freshness    | Age of displayed data    | 95% under 5 min              | App-level monitoring |
| System uptime     | Availability during peak | 99.5% (07–09, 17–20)         | GKE health checks    |
| Response time     | Route query latency      | <2 seconds (p95)             | Prometheus metrics   |
| User satisfaction | In-app rating            | >3.5/5 after 30 days         | Survey + app store   |
| Equity impact     | Cost/km visibility       | 100% of routes show it       | Feature audit        |
| Weather awareness | Rain alert lead time     | >10 min before onset         | IDEAM comparison     |
| Code quality      | Test coverage            | 80% critical, 60% support    | CI pipeline reports  |

**B). Testing Strategy**

We follow a four-level testing approach aligned with CMMI practices:

**Unit testing** (per module): Each NestJS service has unit tests covering business logic. We use Jest with a minimum coverage threshold enforced in CI. The Route Engine module has the highest target (80%) because errors there propagate to user-facing recommendations.

**Integration testing** (cross-module): Tests verify that modules communicate correctly through their defined interfaces. We mock external APIs (TransMilenio, SDM, IDEAM) using recorded responses to ensure deterministic test behavior. Integration tests run on every pull request.

**System testing** (end-to-end): A staging environment mirrors production configuration and connects to sandbox versions of external APIs. We run smoke tests after every deployment and load tests weekly during development, simulating the 07:30–08:15 peak load (128 arrivals/15min, roughly 14,000 events/minute).

**User acceptance testing (UAT):** During the beta phase (Week 14–20), a group of 50 volunteer students will use the app on their daily commutes and provide structured feedback through in-app surveys. UAT success criteria: 70% of testers rate the app 3.5/5 or higher.

**C). Review and Audit Process**

Code reviews are mandatory on every merge request (minimum one peer reviewer). Architecture Decision Records (ADRs) document every significant design change. Sprint retrospectives include a quality review where we check metrics against targets and adjust priorities if any metric is trending below threshold.

**D). Continuous Improvement**

Following ISO 9001’s PDCA cycle: at the end of each sprint, we evaluate quality metrics, identify root causes for any failures, implement corrective actions in the next sprint, and verify their effectiveness. This loop connects directly to the risk monitoring process described in the next section.

**V. RISK MANAGEMENT PLAN****A). Framework**

The risk management approach follows ISO 31000:2018 [10] for the overall framework and PMBOK 7th Edition [11] for the project-level risk register. Risks are identified, assessed (probability  $\times$  impact), treated (avoid, mitigate, transfer, accept), monitored, and reviewed each sprint.

**B). Risk Register****Table 3:** Risk Register with Mitigation Strategies

| ID  | Risk                           | Prob. | Impact | Level    | Mitigation                                                                    |
|-----|--------------------------------|-------|--------|----------|-------------------------------------------------------------------------------|
| R1  | External API downtime          | H     | H      | Critical | Circuit breakers (Kong), Redis cache fallback 60s, static GTFS backup         |
| R2  | Rain disrupts routing accuracy | H     | H      | Critical | Weather as first-class input, IDEAM 5-min polling, mode-shift alerts          |
| R3  | Peak demand exceeds capacity   | H     | M      | High     | K8s HPA auto-scaling, pre-computed route cache at 06:00                       |
| R4  | Low user adoption              | M     | H      | High     | Beta testing with 50 students, WhatsApp integration, iterative UX             |
| R5  | Data freshness below threshold | M     | M      | Medium   | Staleness indicator in UI, 30s TTL on bus positions, monitoring alerts        |
| R6  | Security breach / data leak    | L     | H      | High     | OAuth 2.0, JWT 15-min TTL, TLS 1.3, AES-256, Ley 1581 compliance              |
| R7  | ACO parameters suboptimal      | M     | M      | Medium   | A/B testing in beta, dynamic $\alpha/\beta$ tuning, fallback to cached routes |
| R8  | Team member unavailability     | M     | M      | Medium   | Cross-training, documented ADRs, pair programming on critical modules         |
| R9  | Budget overrun                 | L     | M      | Low      | GCP academic credits (\$5k/yr), monthly cost review, auto-scaling down        |
| R10 | Scope creep                    | M     | M      | Medium   | Sprint backlog discipline, product owner sign-off on new features             |

**C). Risk Monitoring**

Each risk has an assigned owner (see team structure in Section VII) and a monitoring trigger. R1 (API downtime) is monitored by automated health checks every 30 seconds. R2 (rain disruption) triggers when IDEAM reports precipitation above 2.0 mm/h. R3 (peak demand) triggers when HPA scales beyond 8 replicas. R4 (adoption) is reviewed biweekly during beta using download and active-user counts.

**D). Contingency Protocols**

For Critical risks (R1, R2):

If all five external APIs go down simultaneously (worst case), the system enters “offline mode”: static GTFS schedules, cached weather from the last valid reading, and a banner informing users that real-time data is unavailable. Route calculation still works using the last-known graph weights. This

mode was designed to prevent total failure, not to provide optimal routing.

If rainfall exceeds 5.0 mm/h (severe), the system automatically suppresses cycling routes, extends all ETA estimates by the 63% factor measured in W1, and pushes a mass notification to subscribed users with a 10-minute lead time (when IDEAM forecasts allow).

## VI. IMPLEMENTATION STRATEGY

### A). Phased Deployment

The 20-week roadmap from W2 is retained with one modification: we added explicit quality gates between phases. No phase begins until the previous phase's acceptance criteria are met.

**Phase 1 (Weeks 1–4): Foundation.** GKE cluster, CI/CD pipeline, PostgreSQL/PostGIS, Redis, User Management with UDFJC SSO. *Quality gate:* all services healthy, CI deploys to staging in <5 min, authentication flow passes integration tests.

**Phase 2 (Weeks 5–8): Data Integration.** GTFS-RT, SDM traffic, IDEAM weather, IDU construction, campus schedules. Traffic Monitoring with rain classifier. *Quality gate:* all 5 APIs integrated, data freshness <60s for real-time sources, rain classifier accuracy >85%.

**Phase 3 (Weeks 9–14): Core Product.** ACO route engine, travel time estimation, multimodal integration, notifications, mobile app MVP. Beta release at Week 14. *Quality gate:* route calculation <2s (p95), ETA accuracy  $\pm 3$  min for 80% of test routes, beta app installable on Android and iOS.

**Phase 4 (Weeks 15–20): Hardening and Launch.** Predictive models, personalization, community alerts, accessibility (WCAG 2.1 AA), security audit, production deployment. *Quality gate:* 99.5% uptime over 72-hour burn-in, zero critical security vulnerabilities, UAT approval from  $\geq 70\%$  of beta testers.

### B). Change Management

All changes to requirements, architecture, or scope after Phase 1 require a Change Request Form reviewed by the project lead and the affected module owner. Emergency changes (e.g., API format change breaking data integration) can bypass the form but must be documented within 24 hours. This process is lighter than what ISO 9001 would prescribe for a large organization, but appropriate for a 5-person team.

### C). Training and Documentation

User-facing documentation: an in-app onboarding tutorial (3 screens), a FAQ page, and a feedback form. Developer documentation: architecture decision records, API specifications (OpenAPI 3.1), and a contribution guide in the GitHub repository. No formal training sessions are planned for end users because the app is designed to be self-explanatory (validated through UAT).

## VII. PROJECT MANAGEMENT PLAN

### A). Project Charter

**Table 4:** Project Charter Summary

|                         |                                                                                                                                                       |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Project Name</b>     | Urban Mobility Optimization System (UMOS)                                                                                                             |
| <b>Objective</b>        | Deliver a multimodal route planning mobile app for the UDFJC community, reducing average commute uncertainty and making weather-aware recommendations |
| <b>Scope</b>            | Mobile app (Android/iOS), backend microservices, ACO route engine, 5 external API integrations, community reporting layer                             |
| <b>Out of Scope</b>     | Physical infrastructure changes, fare policy modifications, TransMiCable operations, Metro integration (deferred)                                     |
| <b>Success Criteria</b> | Route ETA within $\pm 3$ min (80%), 99.5% uptime during peak, >500 active users in first month, user satisfaction >3.5/5                              |
| <b>Timeline</b>         | 20 weeks (Semester 2026-I)                                                                                                                            |
| <b>Budget</b>           | ~\$780 USD/month infrastructure (GCP credits) + \$0 labor (academic project)                                                                          |
| <b>Sponsor</b>          | Systems Analysis & Design course, UDFJC Computer Engineering                                                                                          |
| <b>Key Stakeholders</b> | 25,700+ UDFJC community, TransMilenio S.A. (data provider), SDM (data provider), course instructor                                                    |

### B). Team Structure and Roles

**Table 5:** Team Structure and Responsibilities

| Role                        | Member     | Responsibilities                                                                  | Phases |
|-----------------------------|------------|-----------------------------------------------------------------------------------|--------|
| Project Lead / Scrum Master | Natalie M. | Sprint planning, stakeholder communication, risk monitoring, quality gate reviews | All    |
| Backend Lead                | Edilson S. | NestJS modules, API design, database schema, ACO implementation                   | All    |
| Mobile Lead                 | Juan E.    | React Native app, Mapbox integration, UX design, Expo builds                      | 3–4    |
| Data Engineer               | Marlon R.  | ETL pipelines, GTFS parsing, TimescaleDB, weather integration, ML models          | 2–4    |
| DevOps (shared)             | All        | GKE management, CI/CD, monitoring, security (rotated weekly)                      | All    |

**Accountability framework:** Each module has a single owner responsible for its quality metrics. The Backend Lead owns the Route Engine and Policy Engine. The Data Engineer owns Data Integration and Traffic Monitoring. The Mobile Lead owns the Presentation layer and Notifications. Persistence is co-owned by Backend Lead and Data Engineer. The Project Lead has veto authority on scope changes and is the single point of contact for the course instructor.

### C). Project Timeline

The timeline follows the 4-phase structure with 10 two-week sprints. Table 6 lists key milestones.

**Table 6:** Key Milestones and Dependencies

| ID | Milestone                  | Week | Depends On | Owner   |
|----|----------------------------|------|------------|---------|
| M1 | Infrastructure operational | 2    | –          | All     |
| M2 | Auth flow complete         | 4    | M1         | Edilson |
| M3 | All 5 APIs integrated      | 6    | M1         | Marlon  |
| M4 | Rain classifier deployed   | 8    | M3         | Marlon  |
| M5 | ACO engine v1 deployed     | 10   | M3, M4     | Edilson |
| M6 | Mobile app beta release    | 14   | M5, M2     | Juan    |
| M7 | Prediction models live     | 18   | M5, M4     | Marlon  |
| M8 | Security audit passed      | 19   | M6         | All     |
| M9 | Production launch          | 20   | M6, M7, M8 | Natalie |

**Critical path:** M1 → M3 → M4 → M5 → M6 → M9. The critical path runs through data integration and the route engine. Any delay in API integration (M3) pushes the entire project. This is consistent with our risk assessment: R1 (API dependency) is the highest-rated risk, and it sits directly on the critical path.

**Gantt overview:** Phase 1 (W1–4) and Phase 2 (W5–8) run sequentially because Phase 2 depends on infrastructure from Phase 1. Within Phase 3 (W9–14), the route engine and mobile app development run in parallel, converging at M6. Phase 4 (W15–20) adds features on top of the beta while hardening for production.

#### D). Resource Management Plan

**Table 7:** Monthly Budget Breakdown

| Item                   | Specification                 | USD/month     |
|------------------------|-------------------------------|---------------|
| GKE cluster (3 nodes)  | e2-standard-4 (4 vCPU, 16 GB) | \$320         |
| Cloud SQL (PostgreSQL) | db-standard-2, 100 GB SSD     | \$140         |
| Redis (Memorystore)    | 2 GB, High Availability       | \$90          |
| Mapbox API             | 50,000 requests/month         | \$100         |
| Networking / CDN       | Cloud Load Balancer + CDN     | \$80          |
| Monitoring             | Prometheus + Grafana stack    | \$50          |
| <b>Total</b>           |                               | <b>~\$780</b> |

Funding source: GCP for Education credits (up to \$5,000/year). At \$780/month, the project can run for approximately 6.4 months on credits alone, covering the full 20-week implementation plus 1 month of post-launch operation. Labor cost is zero (academic project).

**Capacity planning:** During Phase 1–2, all four team members work on backend and infrastructure. During Phase 3, work splits: Edilson and Marlon on backend, Juan on mobile, Natalie on QA and project management. During Phase 4, all hands on hardening and launch. This split aligns with the critical path: backend work (on the critical path) gets 2 engineers throughout, while mobile development (parallel, not on critical path) can tolerate a single developer.

#### E). Communication and Control Plan

**Table 8:** Communication Plan

| Event                | Frequency         | Participants          | Output                |
|----------------------|-------------------|-----------------------|-----------------------|
| Daily standup        | Daily (15 min)    | All team              | Blockers identified   |
| Sprint planning      | Biweekly          | All team              | Sprint backlog        |
| Sprint review        | Biweekly          | Team + instructor     | Demo + feedback       |
| Sprint retrospective | Biweekly          | All team              | Process improvements  |
| Stakeholder update   | Monthly           | Team + instructor     | Progress report       |
| Risk review          | Biweekly          | Project Lead + owners | Updated risk register |
| Quality gate review  | End of each phase | All team              | Go/no-go decision     |

**Tools:** GitHub Projects for sprint boards and issue tracking. Slack for daily communication. Google Meet for sprint ceremonies. Notion for documentation and ADRs. Grafana dashboards for real-time system monitoring (shared with instructor for transparency).

**Escalation path:** If a risk reaches Critical status or a quality gate fails, the Project Lead has 24 hours to document the issue, propose a corrective action, and communicate it to the course instructor. If the corrective action requires a scope change, it goes through the Change Request process described in Section VI.

### VIII. SUCCESS METRICS AND EVALUATION

Post-launch evaluation will use five categories of metrics, measured during the first 30 days of production operation:

**Performance:** Route query latency (target: <2s p95), system uptime during peak (target: 99.5%), data freshness (target: 95% of readings under 5 min).

**Accuracy:** ETA prediction error (target:  $\pm 3$  min for 80% of trips), rain alert lead time (target: >10 min), mode recommendation relevance (measured via user feedback).

**Adoption:** Total downloads (target: 500 in 30 days), daily active users (target: 200), session duration (target: >2 min average), retention rate (target: 40% at day 30).

**Satisfaction:** In-app rating (target: >3.5/5), App Store/Play Store rating (target: >3.5), feature-level satisfaction survey at day 30.

**Equity:** Percentage of routes from southern localities that show cost/km comparison (target: 100%), average routing quality for Ciudad Bolívar users vs. Chapinero users (target: no statistically significant difference in ETA accuracy).

These metrics will be published in a post-launch report and used to prioritize improvements for a potential second development cycle.

### IX. CONCLUSIONS

Across three workshops, this project went from “there is a transit problem” to “here is a system that could help, and here is how we would build, test, manage, and evaluate it.” Each workshop added a necessary layer. W1 gave us the problem understanding. W2 gave us the technical design. W3 gave us the engineering discipline and management structure to actually execute.

The quality framework is not ceremonial. Every metric in Table III traces back to a finding from W1 or a requirement from

W2. The risk register is not generic. R1 (API dependency) sits on the critical path. R2 (rainfall) is the highest-sensitivity parameter from our sensitivity analysis. R4 (user adoption) reflects the reality that students already have WhatsApp groups that work well enough. These are our risks, not risks copied from a textbook.

The project management plan is sized for what we are: a four-person student team with five months and \$780/month in cloud credits. We did not pretend to have a 20-person team or a six-figure budget. The roles, timelines, and communication processes reflect the actual constraints we operate under.

Whether all of this survives contact with reality is the test that comes next. The quality gates between phases are there precisely because we expect some things to go wrong, and we would rather catch them at week 8 than at week 20. The architecture is designed to degrade gracefully, the methodology is designed to adapt to surprises, and the team structure is designed so that no single person's absence kills the project. That is what robust design means in practice: not perfection, but resilience.

## REFERENCES

- [1] C. E. Shannon, "A Mathematical Theory of Communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948.
- [2] Moovit Insights, "Public Transit Index – Bogotá," 2024.
- [3] D. H. Meadows, *Thinking in Systems: A Primer*. Chelsea Green, 2008.
- [4] M. Dorigo, T. Stützle, *Ant Colony Optimization*. MIT Press, 2004.
- [5] S. Newman, *Building Microservices*, 2nd ed. O'Reilly, 2021.
- [6] Congreso de Colombia, "Ley 1581 de 2012 – Protección de datos personales," 2012.
- [7] ISO, "ISO 9001:2015 – Quality management systems – Requirements," International Organization for Standardization, Geneva, 2015.
- [8] IEEE, "IEEE 730-2014 – Standard for Software Quality Assurance Processes," IEEE Computer Society, 2014.
- [9] CMMI Institute, "CMMI for Development, Version 2.0," ISACA, 2018.
- [10] ISO, "ISO 31000:2018 – Risk management – Guidelines," International Organization for Standardization, Geneva, 2018.
- [11] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. PMI, 2021.
- [12] K. Wiegers, J. Beatty, *Software Requirements*, 3rd ed. Microsoft Press, 2013.
- [13] G. Booch, J. Rumbaugh, I. Jacobson, *The UML User Guide*, 2nd ed. Addison-Wesley, 2005.